Taught Tools and Publisher Licenses in Agent Skills

Morgan Nicholson, Samuel Bodenheimer, Aidan Sullivan
School of Informatics, Computing and Cyber Systems
CS599: Open Source Software
Dr. Igor Steinmacher
November 3, 2026

I. Introduction

A skill file tells a coding agent how to carry out a task, and its instructions can name outside products such as packages, developer tools, and hosted services. GitSkills [1] collects the public GitHub skill files written in Anthropic's agent-skill format as they stood in July 2026. We use it to study which products these files teach agents to use and how the projects that publish those products are licensed.

This paper defines the study population, describes how taught tools are extracted, and presents the first checks of the data and of the extraction models. Both research questions remain open, and Section VII describes the analyses required to answer them. Our code and notebooks are kept in the study repository.[1]

II. Research Questions and Related Work

In this paper a *tool* is a product that a skill teaches the agent to use, such as *npm* in a React skill or Elasticsearch in an e-commerce skill. A tool is not the folder that hosts the skill, the repository that publishes the skill, or a link that credits where the skill came from.

RQ1. *Which tools appear most often in skill files?*
RQ2. *How are the projects that publish those tools licensed, once each license is grouped by the freedom it gives?*

The tools found for RQ1 are the input to RQ2. A tool name carries no license of its own, so for RQ2 the unit of analysis is the project that publishes a taught tool. The license on the repository that hosts a skill describes the skill author's project rather than the tool, so we keep it as a separate field and do not use it for RQ2. A mention in a skill file does not show that anyone reviewed or followed the tool's license, so RQ2 describes what skills point to rather than how people comply.

Xing et al. study what agent skills contain and do [2]. Arceo and Narsing study how tools are exposed and secured in agent systems [3]. Stalnaker et al. surveyed 574 developers about licensing and copyright issues that arise from generative AI for software development [4]. None of these studies reports the licenses of the projects behind the tools that skills name.

III. Dataset

GitSkills was built by Destefanis, Graziotin, Vaccargiu, and Ortu from public GitHub repositories during July 2026 [1]. It holds 3,797,117 skill-file rows from 282,200 repositories. Each row is one occurrence of a file, so a skill copied into several repositories appears once per copy. We use release version 3, published on 11 September 2026, and we downloaded it once and kept it as a local copy, so later updates to the public collection stay out of the analysis. The dataset is shared under the Creative Commons Attribution 4.0 International License, which allows sharing and adaptation as long as proper attribution is given.

The collection has four tables.

- *Skill files*, with one row per file occurrence.
- *Repositories*, with metadata such as star counts and the license string of each repository that hosts a skill.
- *Companion files*, which are the files bundled with each representative skill.
- *Collection notes*, which record how the files were gathered.

Commit history is stored on a fetched skill row rather than in a table of its own.

A skill-file row is identified by its repository name (*repo_full_name*) and its path within that repository (*path*). The fields our study relies on are the file name (*filename*), the location class (*location_class*), which separates the usual *.claude/skills* folder from other skills directories and from all other places, and a content hash (*file_sha*) that identical copies share. A flag (*dedup_primary*) marks one representative row for each distinct text, and only that row stores the text itself (*content*). Another flag (*frontmatter_valid*) records whether the header metadata at the top of the file is valid. Skill rows join the repository table on the repository name, which supplies the hosting repository's star count and license string.

Companion-file rows belong to a representative skill and link to it through its path. Each one names a bundled file or folder (*entry_name*, *entry_type*), gives its size and hash (*entry_size*, *entry_sha*), and, when the text was fetched, stores it in *content* up to 100 KB. In this paper we call that stored text *companion text*. The unit of analysis for both questions starts from one distinct skill text, as Section IV explains.

IV. Method

We analyze the local copy of GitSkills with Python and Polars DataFrames. Each notebook sets one shared random seed when it loads the tables, and every sampled draw we report uses a fixed seed, so a rerun returns the same rows.

[1] https://github.com/nichmorgan/msr-data-challenge-2027

A. Study population

To keep the population small and well defined, a skill stays in the study only when every rule below holds, whatever its body length.

- The text is counted once, so copies of the same skill are left out (*dedup_primary*).
- The header metadata at the top of the file is valid (*frontmatter_valid*).
- The file is named *SKILL.md* or *skill.md*, and its body is plain text rather than JSON or CSS.
- The file sits in the usual skill folder under *.claude/skills* or in another skills directory (*location_class*).

The data checks in Section V used the full tables, so these rules were not applied to them.

B. Taught-tool extraction

Extraction reads only the skill body, never the companion text. We extract names with GLiNER [5], which scores each hit, and a cutoff of 0.8 on that score decides which hits we keep. The cutoff and a target accuracy of 80% are both arbitrary choices for this paper, and the manual validation in Section IV-C checks the cutoff against that target. In the label-only run in Section V, labels alone accepted false positives, and Section VII-A proposes improvements to this step.

C. Manual validation

Every step that relies on a model will go through a human quality gate. Those steps are taught-tool extraction, the choice of an owning repository when a tool is not already linked, and license gaps when a model helps fill them.

We will label the taught tools in a seeded random sample of the population, sized before anyone sees the model output, and report precision and recall against those labels. Precision is the share of extracted tools that are truly taught tools, and recall is the share of truly taught tools the model finds. Two authors will label a subset independently, and we will report their agreement. A stage that falls short will run again with new labels or a new cutoff. Once it meets the target accuracy, we will fill the remaining gaps manually and merge the automatic and manual results, dropping entities that do not exist, before the next stage begins. Repository resolution and model-assisted license steps will use the same gate on their own samples, as Section VII describes.

Extracting every tool manually, or checking every model output, would take about the same effort, and either path would limit how many skills, tools, and repositories we can cover. We will therefore accept a bounded residual error under the sample gate so the study can scale, in the same way a survey estimates from a sample instead of visiting every case. In extraction, the gate is meant to catch a model that marks the hosting folder, the publishing repository, an attribution link, the skill filename, or GitHub inside an address as a taught tool.

V. Results

The results so far are data checks and runs of the extraction models, and none of them answers either question. The checks used the full tables, and the larger model runs read companion text rather than the skill body that extraction uses, so they show kinds of errors rather than how often those errors occur.

TABLE I
KINDS OF NON-BLANK COMPANION TEXT

Kind	Rows	Share (%)
Free text	3,236,628	92.8
JSON object	238,834	6.9
JSON array	10,523	0.3
Other JSON	194	0.0

Other JSON is a JSON value that is a string, a number, a boolean, or null.

A. License labels and companion text

After trimming, lowercasing, and filling blank labels, the repository table held 42 distinct license strings, among them MIT, Apache-2.0, GPL-3.0, and AGPL-3.0. These strings describe the repository that hosts a skill, so they are a data check rather than an answer about the projects that publish taught tools. We also classified the 3,486,179 non-blank companion texts, and Table I shows that almost all of them are free text or JSON objects. Both kinds include scripts and configuration, so companion text is not skill prose, and extraction for the study stays on the skill body.

B. Label-only model run

We applied the label-only model at a score cutoff of 0.8 to a seeded draw of 500 companion texts, each clipped to its first 384 characters. Fig. 1 shows every span found in at least two texts. No bar comes near 500 because the cutoff drops every lower-scoring span, so many texts return no span at all. The chart also shows why a human check is needed. The skill filename is marked as a tool even though it names the document itself, and GitHub, which leads with nine files, can be the site inside an address, the place the skill is published, or an attribution link. None of these labels has a measured error rate.

C. Relation model trials

To try the organization link proposed in Section VII-A, we ran a relation model, GLiNER Relex large, version 0.5,[2] on the same 500 companion texts at a cutoff of 0.6. It linked a tunnel setting to Cloudflare as we would want, but it also tied *python3* to a nearby word and a copied repository address to GitHub. A trial on twelve short skill texts, drawn with its own fixed seed and with a length limit that applies only to that trial, still links the wrong pairs. We read those rows manually, so they show where extraction fails but not how often.

Taken together, these results support keeping extraction on the skill body, and they show that label-only extraction at this cutoff counts names that are not taught tools. They do not support any claim about tool frequency or licenses, since such claims require the validation sample and a run over the full population.

[2]Model card at https://huggingface.co/knowledgator/gliner-relex-large-v0.5.

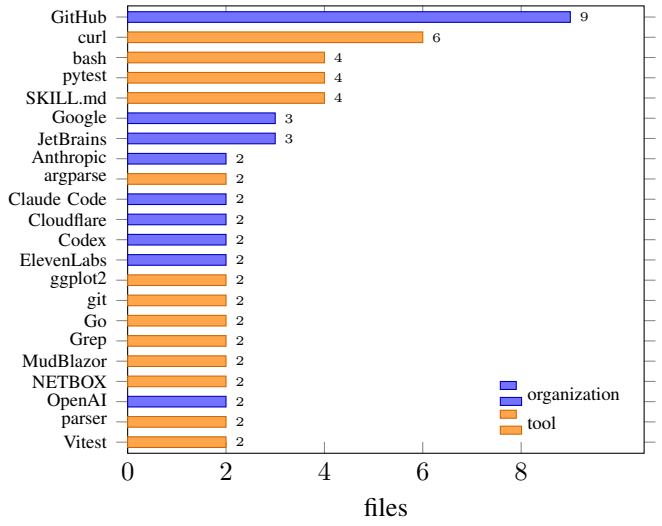


Fig. 1. Spans that the label-only model found in at least two of 500 companion texts at a score cutoff of 0.8, counted as distinct files after folding case. Bars stay small because many texts return no span above the cutoff.

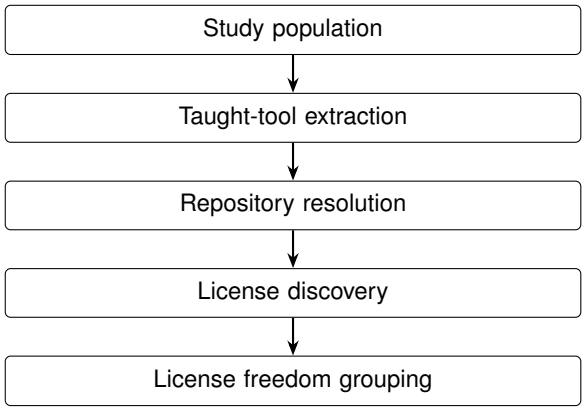


Fig. 2. High-level analysis pipeline. The appendix gives the extended diagram with human checks, manual gap filling, and merges.

VI. Limitations

The main limitation is that no extraction output has been validated against human labels. The validation sample has not been labeled, so no precision, recall, or agreement is reported, and both cutoffs are starting points rather than tuned values. The entity and relation labels for the relation model have not been selected either. The larger model runs read companion text rather than the skill body and clipped each text to its first 384 characters, and like the data checks they ran on the full tables rather than on the study population. Their spans therefore show the kinds of errors to expect, not how often those errors occur in the population.

Another important limitation is deliberate and comes from the original plan. We will check only a random sample of the population that the pipeline processes, so the precision and recall we report will be estimates from that sample, and errors in the rows nobody reviews will remain in the final counts. We accept this to balance the manual effort against an error level we can still trust, because reviewing every row manually would cost about as much as never automating the pipeline, as Section IV-C explains.

VII. Future Work

The analyses below have not been completed. Together with the extraction in Section IV-B, they form the pipeline in Fig. 2, and the appendix gives the extended flowchart with its human checks, manual gap filling, and merges. An extraction rule settled too early would count the wrong tools for the first question and send the wrong set into the second, so they will run in the order below.

A. Relation extraction

Add a relation extraction step that links each tool to the organization that publishes it, when the text names one. Such a link could help confirm that the tool is the central subject of the skill, and the organization could then help find the repository that owns the tool. This step would require a relation model like the one tried in Section V. Its entity and relation labels should filter false positives. A first set would be *tool*, *organization*, and *from_organization*, and other label sets will be compared before one is fixed. The chosen labels will define the extraction setup that the validation sample checks.

B. Manual validation

Label the validation sample, and record precision, recall, agreement, and the errors. The gate, how its sample is drawn, and why we accept a bounded residual error are described in Section IV-C.

C. Full extraction

Extract tools from the full population. Each tool will be counted once per distinct skill file, and names will be folded without regard to case, with the most common original spelling kept as the label. After the quality gate, empty or unresolved rows will be filled by manual analysis and merged before repository resolution. Together these steps will give the frequency list for the first question.

D. Repositories, licenses, and freedom groups

Find the repository for each confirmed tool, read its license, and group licenses by freedom. We will resolve tools one at a time and reuse a project when a tool is already linked to one. Otherwise we will query versioning platforms such as GitHub or GitLab and use a reasoning model, still to be chosen, to select the owning repository from the returned candidates, with open web search as a fallback. A repository address inside a skill can name the skill's own publisher or the source it was copied from rather than the taught tool, so such an address will not be taken as the answer on its own. The same quality gate, manual gap filling, and merge will close this stage before license discovery.

For each resolved repository we will scan the repository root for conventional license file names, such as *LICENSE*

and *COPYING*, read the license identity, and link it to the tool. Gaps will again go through sample validation when a model helps, then manual review, then a merge. The license string stored for a skill's hosting repository will not be used here, because it describes the project that hosts the skill.

Confirmed licenses will be grouped by *freedom*, meaning what the license allows or denies doing with the tool, rather than by SPDX[3] string frequency alone. We will search published sources for a freedom category mapping and fix that mapping before any license is placed in a group. These groups will answer the second question. A named service with no publishing project will stay unresolved, and unresolved cases will be recorded rather than guessed.

These stages carry their own risks. The repository ranking model is still to be chosen, the search may select the wrong repository, a scan of the repository root can miss a license kept elsewhere, and an external service may answer differently on a rerun when it does not honor the seed.

E. Answers

Answer the two questions. Both answers will go in Section V. The first will be the list of tools ranked by the number of distinct skill files that teach them, and the second will be the share of publishing projects in each freedom group, reported next to the number of tools that stay unresolved. The units of analysis follow the pipeline, so the first question will use distinct skill texts and the tools they teach, and the second will use resolved publishing projects that have a freedom group.

APPENDIX A
EXTENDED PIPELINE

Fig. 3 expands Fig. 2 from Section VII. The stages run from top to bottom, and the steps inside each stage run from left to right. A step that relies on a model is followed by a sample check, then either a change of labels or cutoff or a manual review of gaps, and then a merge before the next stage.

REFERENCES

[1] G. Destefanis, D. Graziotin, M. Vaccargiu, and M. Ortu, "GitSkills: A dataset of agent skills on GitHub," in *Proceedings of the 24th IEEE/ACM International Conference on Mining Software Repositories*, ser. MSR '27. Piscataway, NJ, USA: IEEE, 2027, p. To Appear. [Online]. Available: https://arxiv.org/abs/2608.10906

[2] H. Xing, H. Zhuang, X. Zhao, Y. Huang, Z. Tang, and X. Zhang, "Recipes for agents: Understanding skills and their open questions," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, ser. KDD '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 13245–13251. [Online]. Available: https://doi.org/10.1145/3770855.3818652

[3] F. J. Arceo and V. P. Narsing, "Securing the agent: Vendor-neutral, multitenant enterprise retrieval and tool use," in *Proceedings of the ACM Conference on AI and Agentic Systems*, ser. CAIS '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 862–872. [Online]. Available: https://doi.org/10.1145/3786335.3813145

[4] T. Stalnaker, N. Wintersgill, O. Chaparro, L. A. Heymann, M. D. Penta, D. M. German, and D. Poshyvanyk, "Developer perspectives on licensing and copyright issues arising from generative ai for software development," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 2, Jan. 2026. [Online]. Available: https://doi.org/10.1145/3743133

[5] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," 2023. [Online]. Available: https://arxiv.org/abs/2311.08526

[3]Software Package Data Exchange (SPDX) provides short standard license identifiers such as MIT and Apache-2.0. See https://spdx.org/licenses/.

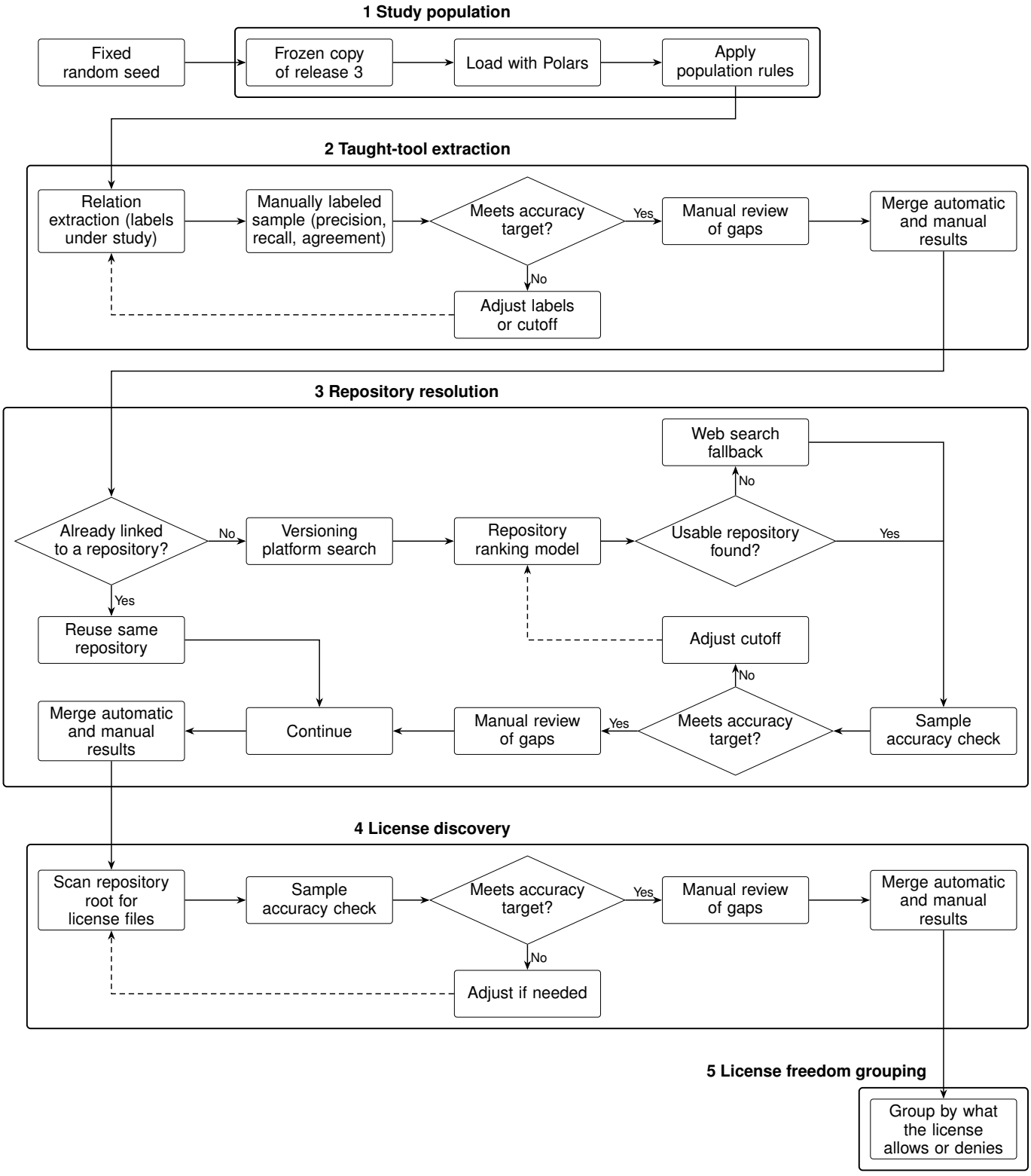


Fig. 3. Extended analysis pipeline. Each stage reads from left to right, except that the second row of repository resolution runs back from right to left. Every exit from a diamond is marked Yes or No. Dashed arrows are retries after a change of labels or cutoff. The extraction labels, the validation sample size, and the repository ranking model are still to be decided.